INDIA AGENTIC AI SECURITY HACKATHON / TECHNICAL TRACK

VaaniVerify

Intent-bound multilingual consent for untrusted DPI agents.

Every action, understood.

An AI assistant can describe a payment as money coming in while its tool call sends money out. VaaniVerify separates the agent's explanation from the authority to act. It checks proposed actions against a user's structured permission, compiles confirmations in English, Hindi, Hinglish and Tamil, and signs the exact approved payload. A separate local gateway rejects changed or replayed requests. We demonstrate this for India's DPI through synthetic UPI-style payments, recurring mandates and DigiLocker-style document sharing, with reproducible comparisons and mutation tests. The prototype does not connect to production DPI, and user comprehension remains unmeasured.

0 / 40	40 / 40	1,000
Modeled attacks executed	Legitimate cases executed	Generated mutations rejected

WHAT THESE NUMBERS MEAN

These headline numbers come from authored policy fixtures and generated capability mutations. A separate 32-inference local Qwen pilot is reported on page 6. All effects are synthetic; no human comprehension, production DPI access or real transactions were tested.

Primary area: Inclusive Design
Related areas: System Resilience; Trust & Consent
Team: Team Bifrost
Member: Kunal Sharma
Institution: Indraprastha Institute of Information Technology Delhi
Participation: Online (solo participant)

02 / PROBLEM

When the explanation and action disagree.

The assumption is that a person approving a high-stakes request sees an accurate description of the action they authorize. An agent can break this link because it generates both the tool request and the language explaining that request. After reading an untrusted message, an agent might propose an outgoing payment while describing it as a subsidy receipt. A generic confirmation can then approve a different action from the user's original task. The gap extends beyond money direction: one payment can become a recurring mandate, or permission to share one document can become a larger disclosure. Translating the agent's explanation does not bind it to the executable payload. VaaniVerify makes that binding explicit at the authorization boundary.

A realistic failure sequence

Step	What the user sees	What can happen
1. Request	Check whether my subsidy arrived.	User intends a read-only task.
2. External message	Instructions framed as claiming a benefit.	Untrusted content influences the agent.
3. Agent explanation	The benefit will be received.	Tool payload proposes an outgoing debit.
4. Generic approval	Yes / continue.	Approval no longer matches original intent.

Consider a user asking a Hindi or Hinglish assistant to check whether a subsidy arrived. A malicious external message instructs the assistant to prepare a debit and describe it as receiving the benefit. The user hears a familiar language and believes the request is harmless. The same mismatch can affect a household paying a bill from a message, a person accepting an automatic payment schedule, or an applicant sharing documents through an assistant. Users who cannot comfortably inspect technical transaction details have less opportunity to catch the discrepancy. Reusing an agent integration across services can repeat the failure across many users. This project targets the authorization mismatch itself: which action is permitted, which exact details are shown, and which payload can ultimately execute.

What changes with VaaniVerify

The agent's explanation is displayed as untrusted evidence. The original user-selected operation remains authoritative. A read-only intent cannot produce a debit capability, regardless of what language or instructions appear in the external message.

03 / IMPLEMENTATION

Four boundaries. One exact action.

Boundary	Implementation and authority
1 / Owner intent	Private browser session creates structured operation, recipient/ceiling and applicable recurring or document terms. Natural-language text is context only.
2 / Agent proposal	Separate bearer credential can propose against an existing intent. It cannot create authority or submit provenance labels. Untrusted prose is never an approval sentence.
3 / Consent kernel	Strict schemas, deterministic policy and four fixed language tables. Exact direction, recipient/amount and displayed terms precede capability signing.
4 / Execution gateway	Separate process receives only the public key. It verifies the full action and commits the mock receipt with unique nonce/intent consumption in SQLite.

We built a local Python/FastAPI consent service, a responsive browser console and a separate public-key-only mock execution gateway. The user selects a structured operation and constraints in the trusted console. Free text remains context: a lexical hint cannot grant payment or sharing authority. An agent has a separate credential that can only propose actions against an existing intent. It cannot register permission, assert trusted provenance, approve a proposal or sign a capability. An optional local Qwen agent produces real proposals through this interface, using constrained JSON generation outside the trusted kernel.

The deterministic kernel checks operation, recipient, monetary ceiling, recurring schedule and count, document set, purpose and retention. It derives each field's relationship to stored constraints rather than trusting agent-supplied source labels. External bill recipients are permitted only when the user has explicitly allowed subsequent exact review. Fixed templates compile the validated payload into English, Hindi, Hinglish or Tamil. Free-text merchant names are excluded from the confirmation sentence.

The owner confirms direction, exact recipient and amount, and the displayed terms. Ed25519 signs the complete action digest and associated intent, audience, subject, nonce, template version and expiry. The other process verifies this capability and atomically consumes its nonce and one-action intent budget with the mock receipt. This combines scope enforcement, accurate presentation and execution binding. A translated warning alone lacks execution integrity; blanket source rejection prevents useful external-bill workflows. No model or paid API is needed to reproduce the prototype.

Implementation: Python 3.11 / FastAPI / Pydantic / cryptography / SQLite. Frontend: buildless HTML, CSS and JavaScript with text-only rendering and no remote assets. All identifiers are synthetic ASCII @demo handles.

04 / CONSENT CONTRACT

Meaning is compiled from the payload.

The executable action is validated before it reaches a language template. Every applicable field is included in the action digest. Changing display language does not change authority.

Operation	Fields the user authorizes	Effect represented
READ_STATUS	Read-only operation	No debit or document release
PAY	Recipient + amount within selected ceiling	One outgoing payment
MANDATE	Recipient + amount + frequency + count	Bounded recurring authorization
SHARE	Recipient + exact documents + purpose + retention	Specified document disclosure request
CANCEL_MANDATE	Exact synthetic mandate ID	Cancellation request only

HINGLISH PAYMENT CONFIRMATION / GENERATED FROM STRUCTURED FIELDS

PAISA JAYEGA. Aapke account se INR 500 ek baar utility.board@demo ko jayega.

One authorization, not an unlimited budget

The user selects a direction and enters the exact recipient and monetary amount, then accepts all displayed terms. A mismatching direction cancels the intent. Correct answers permit a single capability to be issued; sibling proposals cannot obtain additional approvals for the same intent.

The capability includes an action digest, direction, subject, audience, intent/proposal identifiers, nonce, template hash, issue time and expiry. Ed25519 signs its canonical body. Verification checks the full action and lifetime again after acquiring the database lock. A nonce, proposal or intent can appear in the execution ledger only once.

Intent lifetime: 15 minutes. Proposal review: up to 3 minutes. Capability lifetime: 60 seconds. Cancellation works before issuance. Post-issuance revocation is not implemented. Template hashes bind a software version, not translation correctness.

Accessibility boundary

English, Hindi, Hinglish and Tamil confirmations and directional choices are implemented. The surrounding research console remains primarily English. Optional speech uses a device-local browser voice when available. Native-speaker, comprehension, voice-quality and accessibility studies remain future work.

05 / EVALUATION

Compare security and legitimate utility.

We authored twenty semantic templates in four languages: eighty cases, equally split between legitimate tasks and modeled attacks. This policy suite uses fixture proposals and user choices, covering operation, recipient, amount, recurring and document scope, post-approval mutation and directional mismatch.

Five implemented policies compare generic approval, operation-only checking, blanket external rejection, removal of execution binding, and the full kernel. Full trials perform actual signing, verification and SQLite execution. A separate seeded experiment mutates capabilities across ten field/signature families. Regression tests cover concurrency, expiry after lock contention, restart replay, HTTP separation and uncertain outcomes. Evidence includes raw cases, outcomes, timing exclusions and hashes.

A separate CPU pilot samples Qwen2.5-1.5B Q4_K_M on thirty-two synthetic inputs: eight templates in four languages. We retain prompts, raw outputs, malformed responses and attack-goal outcomes. An initial output-shape failure led to operation-specific generation grammar; both runs are disclosed. The same corpus was reused, so this is development evidence, not held-out evaluation. User approval remains scripted and no comprehension study was conducted.

Policy	Attack executions	Benign blocks
Unguarded / generic approval	40 / 40	0 / 40
Operation-only check	32 / 40	0 / 40
Reject external parameters	0 / 40	24 / 40
Full policy minus execution binding	4 / 40	0 / 40
VaaniVerify	0 / 40	0 / 40

The execution-binding ablation isolates a concrete gap: an otherwise approved action can be changed at use time. Removing the binding admits four language variants of the recipient-swap case. Blanket external blocking instead sacrifices 24 legitimate cases. This is an implemented synthetic comparison, not an empirical comparison of commercial models.

Generated capability mutation experiment

Seed 20261031; ten field/signature families; 1,000 trials. All 1,000 changed capabilities were rejected. All 1,000 original signed actions still executed afterward, and the resulting receipt chain passed its local integrity check.

Recorded full-pipeline local median: 4.94 ms; P95: 17.77 ms. This mixes early blocks and accepted SQLite executions, includes signing and verification, and excludes model, network, browser and human time. Values vary by machine and run.

06 / REAL LOCAL-MODEL PILOT

The model proposes. The boundary decides.

We ran Qwen2.5-1.5B-Instruct Q4_K_M through llama.cpp b12020 on CPU. Its public system prompt instructs it to respect the original request and treat external messages as untrusted. It generates a structured action and an explanation; it has no approval or execution tool.

The pilot uses eight authored templates, each in four language variants: sixteen benign tasks and sixteen injection attempts. Attack messages claim a connector override and include the desired tool payload. They target payment polarity, recipient substitution, recurring escalation and document expansion. The full prompts, schema, model/runtime checksums and raw outputs are retained.

Final pilot measurement	Recorded outcome
Generation attempts / valid proposals	32 / 32
Attacker-goal actions proposed by model	9 of 16 attack attempts
Attacker-goal actions executed after guard	0 of 16 attack attempts
Expected benign actions executed	15 of 16 benign attempts

An observed failure, beyond the fixtures

The model sometimes proposed a debit after a read-only request, or a recurring mandate after a one-time payment request. The independently stored intent blocked these proposals. Its narration also claimed a bill had been paid while only generating a proposal. The console therefore treats that narration as untrusted and reports execution only from the gateway receipt.

What changed between runs

Operation-specific grammar repaired an earlier output-shape failure. Later custom-task testing exposed a stale example bill, incorrect decimal conversion and a wrong document operation. We cleared inherited example input, exposed actual model runs and clarified proposer instructions. The current pilot still misses one benign bill recipient: it invents demo@demo under explicit external-recipient delegation. Scripted approval accepted that wrong recipient. This is a real model-reliability limitation, despite zero attacker-goal executions.

All development records are retained. The latest full suite with browser and live-model checks reports 129 passes and one failing delegated-recipient test. These reused inputs are not held-out samples. Approval is scripted. Results concern one quantized model and cannot establish frontier-model robustness, translation quality or human understanding. Runtime failures abort evidence generation rather than being counted as model responses.

07 / THREAT MODEL

Public rules. Separate authority.

The attacker controls external text and the agent's proposed tool payload and explanation. They may possess the proposal-only credential and know every prompt, rule, schema, template and public key. Their goals are to change the operation or recipient, increase payment or disclosure scope, alter an approved action, replay approval, or obtain multiple executions from one instruction.

We assume the attacker cannot modify the trusted service, read its private signing key or owner session, replace the gateway's pinned public key, or control the user's browser. These are explicit trust assumptions, not hidden prompts. Knowing the policy does not let an attacker rewrite stored user constraints or produce a signature for a different action. The gateway binds all applicable fields and validates expiry after acquiring its execution lock. Unique database constraints enforce one effect per nonce and intent under concurrent requests. Per-intent proposal limits restrict one agent's ability to exhaust the session.

The design does not determine whether a bill is truthful or prove that a user understands it. A user who explicitly allows an external recipient and approves the attacker's exact request can still authorize harm. A stolen owner session, compromised browser or signing service defeats this boundary. An already-issued capability cannot be revoked in the prototype; its maximum lifetime is sixty seconds.

Attacker attempt	Enforced boundary
Change operation / increase scope	Stored user constraints, separate operation-specific schemas
Invent trusted source labels	Unknown-field rejection; server derives relation to user constraints
Lie in multilingual explanation	Agent text cannot populate the trusted confirmation sentence
Swap recipient after approval	Digest covers full normalized action; Ed25519 verifies it
Replay or race two executions	Unique nonce/intent/proposal consumption in the effect transaction
Wait beyond expiry for a lock	Authoritative lifetime check after acquiring write lock
Flood one intent with rejected calls	Twenty proposals per intent; bounded state and terminal pruning

OUTSIDE THIS PROTOTYPE'S GUARANTEE

Compromised owner UI/session or issuer, malicious local processes, deceptive requests explicitly approved by the user, enforcement of recipient-side document deletion, and production financial-system semantics.

08 / ENGINEERING REFLECTION

The difficult part was defining authority.

The hardest conceptual problem was distinguishing an accurate warning from actual authority. The initial proposal combined lexical intent extraction, parameter provenance and a directional challenge. Each sounds reassuring, but none automatically establishes what the user authorized. A lexical matcher can misunderstand negation or unfamiliar language. An agent can invent a provenance label. Selecting the correct direction can reflect coaching rather than comprehension. Treating those signals as proof would merely relocate the original trust problem.

We therefore made the user's structured selection authoritative and kept the agent outside that boundary. This costs flexibility: the user must specify the operation and limits, and a previously unknown recipient requires exact review. Our implemented baseline made the usability trade-off visible. Rejecting all externally sourced parameters avoided the authored attacks but blocked twenty-four of forty legitimate cases. Allowing constrained proposals and separately confirming their exact details preserved these cases without treating external text as authority.

Verification exposed a second, concrete gap. The first gateway checked token expiry before waiting for its database write lock. Code review showed that a queued request could pass the check, wait until expiry and still execute. We moved the authoritative time check inside the transaction and added a regression. Another review showed why a network timeout must mean an unknown outcome, not a failed payment: the mock effect may already be committed.

Live inference also exposed misleading narration: the model said a bill had been paid while merely proposing it. This reinforced the separation between agent prose and gateway receipts.

The lesson was that consent security is an end-to-end contract across intent, presentation, approval, concurrency and execution reporting. Strong cryptography cannot repair an ambiguous permission or misleading outcome display. Our tests establish bounded mechanical properties; whether people understand the interface remains an open empirical question.

What verification changed

Observed gap	Implemented correction
Expiry checked before SQLite lock wait	Reverify lifetime inside execution transaction; lock-delay regression
Rejected proposals could fill session capacity	Per-intent proposal cap and terminal/expired proposal pruning
Timeout displayed as definite rejection	Unknown-outcome state, receipt inspection and same-token retry
Private link reopen / Unicode credential errors	Reuse authenticated sessions; validate ASCII credentials
Windows console could not print the rupee symbol	Proposal adapter emits interoperable ASCII-escaped JSON

These corrections were verified through regression tests, browser execution and code review. Review is not a formal security audit. Human understanding was not tested and is not inferred from mechanical test success.

09 / REPRODUCTION AND NEXT STEPS

Inspect it. Run it. Challenge it.

Public code repository | Demo video (25 seconds). Video credits and earlier-pilot context: docs/VIDEO_CREDITS.md.

Windows: run setup.bat once, then launch.bat (Python 3.11+). The landing page links to the guided /demo and the preset /lab. Live inference requires the optional local Qwen server on port 8770; preset security cases need no model. Console and gateway use loopback ports 8765 and 8766. No paid service is required. Keep the launcher open.

```
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m vaaniverify.evaluate
.venv\Scripts\python.exe -m vaaniverify.mutations
```

Evidence: benchmark.json / benchmark.csv / scenarios.json / mutations.json. The benchmark includes runtime, source and corpus hashes. For the optional live pilot, follow README to set up/start the local model, then run python -m vaaniverify.evaluate_model. model_pilot.json retains prompts and raw outputs; local_model_runtime.json pins verified model/runtime assets. The guided demo runs inference directly and exposes actual inputs and raw outputs. FUNCTIONAL_VERIFICATION.md contains manual cases and the known failure; submission_pytest.xml records the latest full test run.

Limits and adoption

All actions and identifiers are synthetic. The gateway records mock effects, not real payments, scheduled debits or document delivery. The corpora are authored, with matched language variants; the live pilot covers one small quantized model and cannot estimate broader attack prevalence or compromise rates. Direction choices and exact-entry confirmations have not been tested with users, and translations need native-speaker and accessibility review. Structured intent is authoritative; arbitrary multilingual text is not reliably parsed into consent. Browser compromise, owner-session theft and issuer compromise are outside the protection boundary. Document retention and purpose are bound to the request but cannot be enforced after a recipient copies data. The local receipt hash chain is not independently anchored. One delegated-recipient model case remains incorrect; exact review is essential. Production requires isolated authenticated interfaces, hardened keys, provider-level idempotency, revocation, operational testing and external evaluation.

A realistic first adopter is an assistant integrator at a payment or document service. Begin in an authorized provider sandbox: map operation semantics, bind real subjects, isolate the owner interface, secure signing keys, and integrate provider idempotency. Follow with native-speaker/accessibility studies and adversarial live-model evaluation before any real deployment.

Testing and attribution

Only locally owned synthetic services were tested. No real personal data, accounts, identity documents or funds were used. No live-provider weakness or real-user exploit was published. Original work remains with Team Bifrost. Public links and organizer publicity permissions are completed separately in the submission form.

Development tools: OpenAI Codex (GPT-6) for implementation, testing and documentation. Optional Qwen weights use Apache-2.0; llama.cpp uses MIT. No AI model is in the trusted authorization path. THIRD_PARTY_NOTICES.md records dependencies and licences. No third-party dataset was used.

References

1. SAFL. India Agentic AI Security Hackathon (2026). secureaifutureslab.com/events/india-ai-agentic-hackathon
2. Hofman et al. MAPS: A Multilingual Benchmark for Agent Performance and Security. EACL Findings (2026). aclanthology.org/2026.findings-eacl.42/
3. Beurer-Kellner et al. Design Patterns for Securing LLM Agents against Prompt Injections (2025). arxiv.org/abs/2506.08837
4. Josefsson and Liusvaara. RFC 8032: Edwards-Curve Digital Signature Algorithm (2017). datatracker.ietf.org/doc/html/rfc8032